PreCap v0.3: A Depth-Calibrated Activation with a Soft Fixed Bound for Deep Normalization-Free MLPs — Training Depth and 4-Bit Robustness in One Form

Aaron Schmitt
Independent researcher, Wiesbaden, Germany · ORCID 0009-0009-8689-5848
Preprint, version 0.3 · November 2026 · Code and data: github.com/hermi-labs/precap-activation

Abstract

Version 0.2 found that no member of the PreCap family is both trainable at depth and robust to 4-bit activation quantization: calibrating the gradient gain $\chi 1 = 1 + k/L$ for depth widens the output range, and the wide range costs accuracy at 4 bits. Version 0.3 removes this trade-off by putting the depth-calibrated PreCap core inside a soft, fixed bound. The resulting form, BPreCap, is the identity on most of its input range and approaches $\pm M$ smoothly ($M = 3$); its constants are solved so that the whole function, bound included, keeps $\mathrm{E}[f^2] = 1$ and $\chi 1 = 1 + 2/L$. We report 16 preregistered, hash-frozen studies and one descriptive diagnosis on fully connected networks with 100 and 200 layers, without normalization layers and without skip connections (MNIST, FashionMNIST, CIFAR-10, CIFAR-100; paired Wilcoxon tests with Holm correction; post-training quantization with weights at 8 bits and activations at 4 bits).

(1) **Depth and 4 bits in one form.** At depth 200 on MNIST, BPreCap keeps a 4-bit test error of 14 % where the depth-calibrated member of v0.2 reaches 77 %, at a fp32 cost of about 1 pp.

(2) **Against TAT under the same recipe.** With the recipe of Tailored Activation Transformations (orthogonal initialization and per-example input normalization), BPreCap is on par with TAT in fp32 on CIFAR-10 and better at asymmetric 4 bits: −3.8 pp with Adam and early stopping, −4.5 pp against TAT with weight decay, and −11.7 pp against TAT at the setting chosen by fp32 tuning (large LR, no weight decay). On CIFAR-100 it is better by 1.9 pp at 4 bits, but here the advantage comes from fp32 accuracy, not from a smaller 4-bit cost. On CIFAR-10 the 4-bit cost of BPreCap stays at about +3 pp in every regime, while TAT's varies from +7 to +15 pp with LR and weight decay. The absolute activation range does not explain this: with weight decay, TAT's range is as small as BPreCap's, and its 4-bit cost is still more than twice as large.

(3) **Baselines and what the bound contributes.** Under the same recipe, ReLU and GELU do not train well at depth 100 (BPreCap −39 and −40 pp at 4 bits), but SELU is a strong baseline at depth 100: only 0.95 pp behind in fp32 and on par at 4 bits, with an equally small 4-bit cost. At depth 200, SELU falls behind: BPreCap is better by 11.97 pp in fp32 and 8.95 pp at 4 bits (12/0 each), and 5 of 12 SELU runs barely train. With the linear core, a soft hull is clearly better at 4 bits than a hard clip at the same bound (−4.7 pp); with the PreCap core, which saturates smoothly by itself, there is no difference. A learnable hard clip in the style of PACT did not learn under AdamW in our setup (its bound moved only through weight decay), so a fair comparison with PACT is still missing. A linear core with the same soft bound (LB) is often as good or better at 4 bits, while the PreCap core is better in fp32 on CIFAR-10 and CIFAR-100.

An independent PyTorch/CUDA implementation on different hardware reproduces the main comparisons with medians within 0.5 pp of the original runs. BPreCap trains only with small learning rates (it diverges at peak LR $2{\cdot}10^{-4}$ where TAT still trains). The principle of depth-dependent activation shaping and the benefit of bounded activations for quantization are known; our contribution is a controlled measurement of one form that combines both. All negative results are reported.

1 Introduction

Self-normalizing networks [1] showed that an activation function alone can keep the forward signal of a deep fully connected network at zero mean and unit variance, without normalization layers. PreCap is a bounded member of this line: an exponential-linear unit behind a soft tanh clip, smooth, monotonic and bounded on both sides. Version 0.1 [34] introduced PreCap and its self-normalizing form SPreCap and found it on par with SELU in 50- and 100-layer MLPs. Version 0.2 [35] calibrated the gradient gain for depth, compared the result with Tailored Activation Transformations (TAT) [12], and measured robustness to 4-bit activation quantization.

Version 0.2 ended with a trade-off. The depth family kLk ($\chi 1 = 1 + k/L$) trains 100- and 200-layer MLPs without normalization, but its output range grows with depth (up to $[-4.2, 12.8]$ at depth 200), and its 4-bit loss rises to 18–31 pp at depth 100. The bounded member SPreCap-cs is robust at 4 bits but does not train at depth 200. Version 0.3 asks whether one form can do both. *Does a soft, fixed bound around the depth-calibrated core keep both properties? How does*

it compare with TAT [12] under the same recipe and under a common training regime? Which part of the form matters: the bound, its softness, or the PreCap core?

Our contributions are:

- BPreCap, a depth-calibrated PreCap core inside a soft bound, with constants solved for the bounded function (Section 2);
- preregistered comparisons with TAT under the same recipe, at two depths, two training regimes and on CIFAR-10 and CIFAR-100 (Section 5.3);
- controlled comparisons of the bound itself: soft against hard at the same bound and the PreCap core against a linear core (Sections 5.2, 5.4), and a negative result for a learnable clip under AdamW;
- baselines with ReLU, GELU and SELU under the same recipe: SELU matches BPreCap at 4 bits at depth 100 but falls clearly behind at depth 200 (Section 5.5);
- a measured dependence of TAT's 4-bit cost on learning rate and weight decay, which the bound removes, and evidence that the absolute activation range is not the mechanism (Section 5.7).
- an independent replication of the main comparisons with PyTorch/CUDA on other hardware (Section 5.6).

2 Definition

2.0 PreCap core (from v0.1/v0.2)

PreCap(x) = $\lambda \cdot (1$ if $x > 0$ else $\alpha) \cdot \mathrm{capexp}(s \cdot \tanh(bx/s))$, with $\mathrm{capexp}(w) = \mathrm{e}\hat{}\,w - 1$ for $w \le c$ and $\mathrm{e}\hat{}\,c - 1 + \mathrm{e}\hat{}\,c(w - c)$ for $w > c$, $s = 1.5$, $c = 1$. capexp alone is a shifted ELU [8]; the tanh clip in front makes the function bounded. SPreCap chooses λ and α so that zero mean and unit variance are a fixed point for standard normal inputs, and b sets the gradient gain $\chi 1 = \mathrm{E}[f'(z)^2]$. The depth family kL$k$ of v0.2 sets $\chi 1 = 1 + k/L$ [35].

2.1 Soft hull (Lagma)

For a bound $M > 0$ and a knee $\rho \in (0, 1)$, the soft hull, which we call Lagma, is

$h_\mathrm{M}(y) = M \cdot \varphi_\rho(y/M)$, $\varphi_\rho(u) = u$ for $|u| \le \rho$, $\varphi_\rho(u) = \mathrm{sign}(u) \cdot (\rho + (1 - \rho) \cdot \tanh((|u| - \rho)/(1 - \rho)))$ otherwise.

h_M is the identity on $[-\rho M, \rho M]$, continuously differentiable (slope 1 on both sides of the knee), strictly increasing and bounded by $\pm M$. We use $\rho = 0.8$ throughout; neither ρ nor M was optimized.

2.2 BPreCap and LB

BPreCap applies the hull to the PreCap core of v0.1/v0.2:

$\mathrm{BPreCap}(x) = h_\mathrm{M}(\mathrm{PreCap_\{\lambda,\alpha,b\}}(x))$, $M = 3$, $s = 1.5$, $c = 1$.

For each depth L, (λ, α, b) are solved jointly so that, for $z \sim \mathrm{N}(0, 1)$, the bounded function satisfies $\mathrm{E}[f(z)] = 0$, $\mathrm{E}[f(z)^2] = 1$ and $\mathrm{E}[f'(z)^2] = 1 + 2/L$ (Table 1). The hull is part of the function when solving, so the calibration holds for the function actually used.

As a control we use $\mathrm{LB}(x) = h_\mathrm{M}(\beta x)$, a linear core with the same hull. For LB, β and M are solved for the same three conditions; this gives a narrower bound than for BPreCap. LB is a smooth, depth-calibrated version of the critical hard-tanh of Pennington et al. [18].

Table 1. Constants ($\rho = 0.8$). Derivation: `math/p4_bounded_depth.py`, `math/p5_linear_bound.py`; tested against the study code to 10^{-12} (`tests/test_v03.py`).

Form	Depth	$\chi 1$	λ	α	b	β	M
BPreCap	100	1.020	2.9844	1.4067	0.2897	–	3
BPreCap	200	1.010	4.9097	1.2489	0.1846	–	3
LB	100	1.020	–	–	–	1.0353	2.2165
LB	200	1.010	–	–	–	1.0187	2.4543

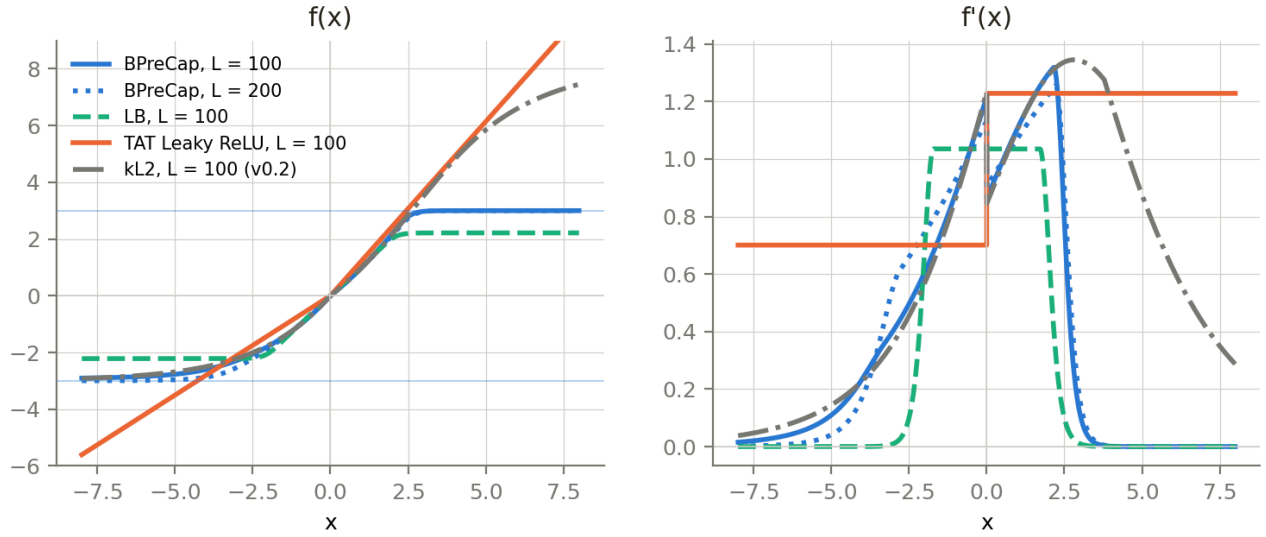


Figure 1 (computed from the package code). BPreCap at depths 100 and 200, LB at depth 100, TAT's Leaky ReLU at depth 100 and kL2 at depth 100 from v0.2. Left $f(x)$, right $f'(x)$. Like SELU, the PreCap core has a jump of the derivative at 0 (slope λb for $x > 0$, $\lambda\alpha b$ for $x < 0$); the hull itself is C^1.

2.3 Why a fixed bound

A fixed nonlinear activation makes inputs indistinguishable with depth (the C-map of every such function converges to a fixed point), which is why depth-dependent shaping as in DKS and TAT is needed [12, 13]. We recompute this for depth 512 in `math/p12_tiefe_kollaps.py`: the distance between two inputs after 512 layers is $8.7{\cdot}10^{-6}$ for ReLU and $3.3{\cdot}10^{-7}$ for SELU, and 0.066 at every depth for a depth-adapted Leaky ReLU. Shaping fixes the kernel at initialization. It does not fix the distribution of the activations after training. Section 5.7 shows that, for TAT, the 4-bit cost after training depends strongly on learning rate and weight decay. With the bound, the activation distribution is limited by the function itself instead of by the training run.

3 Related work

Activation functions and self-normalization. A bounded activation scaled for unit variance is old: LeCun et al. recommend $1.7159{\cdot}\tanh(2x/3)$ [44], whose bound is fixed and whose scale keeps the variance near 1, but which is not calibrated for depth. SELU [1] introduced the mean/variance fixed point; bSELU [2] and gradient-norm equality [17] give depth-dependent tolerances, and Hayou et al. [3] analyze how the activation shapes trainability at depth. Bidirectional self-normalization [4], SaluNet [5] and normalization-free Transformers with bounded activations such as DyT and Derf [6, 7] pursue training without normalization layers. ELU [8], TanhExp [10] and the survey of Kunc and Kléma [11] cover the wider design space; Bukva et al. [9] contrast criticality with uniformity in deep networks.

Signal propagation without normalization or skips. Mean-field theory [3, 9] and dynamical isometry [18] explain why deep networks without normalization are hard to train; Pennington et al. use a critical hard-tanh with orthogonal weights. DKS [13] and TAT [12] shape the activation for a given depth; recent work extends this line to residual-free Transformers [19] and diffusion encoders [20]. DyT [6] replaces LayerNorm in Transformers by a bounded $\tanh(\alpha x)$. None of these studies quantization of the activations.

Bounded activations and quantization. Bounding activations before quantization is old: DoReFa-Net [21] bounds activations to [0, 1], PACT [16] learns a clipping bound per layer, BReLU [22] uses a static range for integer-only CNNs, and CReLU [36] is a clipped ReLU with a learnable bound, designed for binary-weight networks with low-bit activations. ReLU6, a fixed hard clip at 6, is used in mobile networks for its robustness under low-precision computation [46]. Price et al. [45] clip sparsifying activations (shifted ReLU, soft thresholding) at a magnitude prescribed by the Gaussian-process variance map, so that deep networks stay trainable at initialization; this is the closest case of a bound derived from signal-propagation theory that we found, but it is a hard clip chosen for sparsity, not for depth or quantization. Q-MLP [42] uses bounded activations (PACT) together with LayerNorm to quantize MLP-based vision models to 4 bits. These bounds are hard clips in networks with normalization layers, mostly trained with quantization in the loop. Closest to our form is the SoftCap family [37]: rectifiers whose positive branch is a variance-preserving $a{\cdot}\tanh(x)$, which suppress post-activation outliers and degrade less under INT8 quantization in Transformers. Its authors state that these activations rely on normalization layers and leave sub-8-bit quantization for future work; the scale is fixed for variance, not calibrated

for depth. Analytic clipping for post-training quantization [23] and percentile/MSE calibration [15] are standard. Park et al. [24] use a smooth saturation on the *weights* for quantization. He et al. [25] show that outlier features, which make quantization hard, emerge during training and are promoted by large adaptive learning rates; Chavan et al. [39] link such outliers under AdamW and Muon to dominant singular values, and Outlier-Safe Pre-Training [38] prevents them for 4-bit LLMs through the optimizer and a single-scale normalization; Tarawneh [26] finds that the required bit width grows with depth. Ji et al. [27] report that residual-free Transformers quantize better than residual ones. Training-time regularizers that make activations easier to quantize after training, without changing the activation function, have also been proposed, e.g. SCULPT [30]. Catalan-Tatjer et al. [29] show that post-training quantization error depends on the learning-rate schedule and diverges from validation loss once the learning rate decays. Sun et al. [28] note that removing BatchNorm for integer-only inference is rarely stable; WAGE [40] replaced batch normalization by a constant scaling for low-bit integer training, and EOQ [43] removes it for 8-bit integer training with Fixup initialization. InfoQuant [41] argues from information theory that a quantization-friendly activation distribution has a small range and enough dispersion within it, and shapes LLM activations after training by orthogonal transforms. Blumenfeld et al. [31] derive a quantization-depth trade-off for networks with quantized activations at initialization: the maximal trainable depth grows only polynomially with the number of quantization levels. Shkolnik et al. [32] regularize the kurtosis of weights to make models robust to quantization, and Scholten et al. [33] show that rounding errors compound across layers in post-training quantization and add a smoothness penalty that damps this compounding.

Our position. We do not claim that bounding activations helps quantization, nor that activation ranges grow during training; both are known [16, 21, 25]. To our knowledge, the combination measured here has not been reported: a depth-calibrated form with a fixed, soft bound that trains deep MLPs without normalization or skips, compared with TAT and common activations under the same recipe, with controlled comparisons of soft against hard bounds and of the PreCap core against a linear core. Unlike SoftCap [37], the form is calibrated for a given depth, trains without normalization, and is evaluated at 4 bits. This assessment rests on citation tracking of DKS, TAT, PACT (all 1186 citing titles screened by keyword), Park et al. and Ji et al., and on keyword searches. A second round used OpenAlex, which also indexes journals outside the arXiv: the citing works of Pennington et al., Schoenholz et al., SELU, PACT, Blumenfeld et al., Hayou et al. and DyT were screened for the terms quantization, bounded, saturating and low-bit (446 titles), together with twelve free searches (600 titles) and twelve targeted arXiv searches. Google Scholar could not be searched.

4 Experimental protocol

Studies. Version 0.3 reports 16 studies (B1–B15 and B10b), each preregistered and hash-frozen (SHA-256) before it ran, with one change relative to its predecessor. Each study contains a control run that must reproduce an earlier study bit for bit; all controls did, including after an operating-system update between B14 and B15 (checked with an additional rerun from a clean copy). All B-studies ran on Apple MLX on an M1 Pro.

Networks and data. MLPs of width 256 and depth 100 or 200, without normalization layers and without skip connections, batch 128. MNIST and FashionMNIST (B1, B2, B4), CIFAR-10 (B3, B5–B10, B12–B15) and CIFAR-100 (B11), with the last 5000 training images as validation split.

Two initializations. *Gaussian:* as in v0.2. *TAT recipe* (suffix _tr): scaled orthogonal initialization and per-example input normalization to mean square 1, as in [12], for all forms in the comparison.

Three training regimes. (i) Adam with constant LR, 1720 or 6880 steps (B1–B9). (ii) AdamW with cosine decay to zero over 6880 steps, no warmup (B10–B15). LR and, in (ii), weight decay are tuned per form on separate tuning seeds by validation accuracy at the end of training, and then evaluated on 12 disjoint confirmation seeds. (iii) In B9–B15 each run is evaluated at its own best validation checkpoint (every 860 steps, chosen by fp32 validation error; the test set is never used for selection).

Quantization. As in v0.2: simulated post-training quantization, weights int8 per output channel, activations (inputs of every linear layer) int4 per tensor, either symmetric (w8a4) or asymmetric with a zero point (w8a4a), each with the best of four calibrations per run (max, 99.9th and 99.99th percentile, MSE) on 1000 training images. The primary measure from B6 on is w8a4a, the scheme that favors TAT most.

Replication on other hardware (C6). An independent PyTorch implementation of the same procedure, checked against the MLX code before the run (identical data and initial weights; gradients as close to a float64 reference as the MLX gradients; identical AdamW update up to float32 rounding), was run on an NVIDIA RTX 4070 (CUDA 12.6, float32 without TF32, deterministic algorithms) for BPreCap, LB, TAT (with and without weight decay) and SELU at depth 100, 12 seeds each, with the hyperparameters chosen in the MLX studies.

TAT. Leaky ReLU with the parameters of the official `dks` library for η = 0.9 (depth 100: a = 0.5704; depth 200: a = 0.6796), trained with the same optimizer as all other forms.

Statistics. Paired by seed, two-sided exact Wilcoxon signed-rank test, Holm correction over the secondary tests of a study (α = 0.05). A failed run counts as chance-level error. "Better"/"worse" requires a significant result; otherwise "inconclusive". *Note:* with a zero difference, scipy silently switches from the exact test to the normal approximation. This happened in 4 secondary tests (B1, B4 twice, B11). With tied absolute differences, scipy's exact p ignores the ties. An independently written script (`analysis_check/nachrechnen.py`, exact sign-flip test with mid-ranks, no scipy) recomputes the reported tests from the raw CSVs; differences, medians and W/L agree exactly, the p-values differ in these cases only slightly (e.g. B10, fp32: 0.275 instead of 0.266), and no verdict changes.

5 Results

5.1 Depth and 4 bits in one form (B1, B5)

On MNIST and FashionMNIST at depths 100 and 200 (Gaussian initialization, LR from kL2), BPreCap is far more robust at symmetric 4 bits than kL2 from v0.2 in all 4 cells (−14.9 to −62.9 pp, 12/12 seeds each) [B1, S2]. At depth 200 on MNIST, the median w8a4 test error is 14.4 % for BPreCap and 77.3 % for kL2. The price is fp32 accuracy at depth 200 (+1.1 pp in both datasets) [B1, S1]. SPreCap-cs, the robust member of v0.2, does not train at depth 200 (42.7 % on MNIST), while BPreCap reaches 7.1 % [B1].

The linear control LB matches BPreCap in fp32 on MNIST/FashionMNIST (no effect in 4 of 4 cells) and is better at 4 bits (1.8–4.6 pp) [B1, S3/S4]. On these datasets the PreCap core adds nothing; Section 5.4 shows that this changes on CIFAR.

On CIFAR-10 at depth 200 with retuned learning rates, BPreCap's 4-bit cost is +7.7 pp against +23 pp for kL2 [B5].

5.2 Soft against hard bounds

Gaussian initialization (B2, B4). With the same bound and the same criticality, the soft hull is better at symmetric 4 bits than a hard clip: −0.86 to −1.80 pp in 4 of 4 cells for LB against a hard clip solved for the same criticality, whose bound is then slightly wider (2.30 instead of 2.22 at depth 100) [B2], and −0.82 to −1.62 pp (2 of 4 cells significant, the other 2 in the same direction) at exactly the same bound [B4]. The width of the bound alone has no effect [B4, S2/S3]. With a wide bound the effect is not significant [B4, S1].

TAT recipe and AdamW regime (B13). With the PreCap core, the soft hull is **not** better than a hard clip at the same bound in the TAT recipe and AdamW regime (w8a4a +0.04 pp, 6/6, p = 0.34, inconclusive; fp32 −0.40, inconclusive) [B13]. The PreCap core already saturates smoothly through its inner tanh, and the hard clip at 3 is rarely active (1.5 % of values). With the linear core, where the hull is the only nonlinearity, the soft hull is clearly better at the same fp32 accuracy: −4.69 pp at w8a4a, −4.84 pp at w8a4 and −16.96 pp at w8a3 (each 12/0) [B13]. The hard-clipped linear form is piecewise linear and pays +7.7 pp at 4 bits, close to TAT with weight decay (+7.3 pp), while the soft form pays +2.3 pp.

Learnable hard clip, PACT style (B14). BPreCap and a hard clip with a learnable bound per layer (start at M) are on par at w8a4a (+0.04 pp, 6/6, inconclusive); with the linear core the fixed soft hull is better (−4.70 pp, 12/0) [B14]. However, **the bound did not learn**: its change is fully explained by the decoupled weight decay of AdamW (from 3.0 to 2.711, against $3 \cdot \exp(-0.3 \cdot \Sigma \mathrm{lr}) = 2.706$ from weight decay alone; without weight decay it stays at 3.00). B14 is therefore a repetition of B13 with a slightly shrunk bound, not a test of PACT. A fair comparison needs a separate learning rate or SGD for the bound, PACT's L2 penalty, and quantization-aware training. *Limitation stated in advance:* under Adam a parameter moves by at most about the LR per step, so the learnable bound can move by at most about 0.34 (peak LR 10^{-4}) to 1.72 ($5 \cdot 10^{-4}$) over a cosine run. Our version is symmetric, applied after training, and uses AdamW weight decay instead of PACT's L2 penalty; it is a replication of the idea, not of the original method.

5.3 Against TAT under the same recipe (B6, B8, B9, B10, B10b, B11)

Table 2. BPreCap_tr – TAT, median paired difference of test error (pp) and W/L over 12 seeds. Negative favors BPreCap. ES = each run at its best validation checkpoint.

Study	Data, depth	Regime	fp32	w8a4	w8a4a (primary)
B6	CIFAR-10, 100	Adam const., 1720 steps	+1.01 (worse)	−2.59 (12/0)	−0.76 (10/2)
B8	CIFAR-10, 200	Adam const., 1720 steps	+0.30 (inconcl.)	−3.34 (12/0)	−0.98 (11/1)
B9	CIFAR-10, 100	Adam const., 6880 steps, ES	+0.20 (inconcl.)	−6.58	−3.76 (12/0)
B10	CIFAR-10, 100	AdamW cos., tuned WD, ES	−0.21 (inconcl.)	−14.76	−11.67 (12/0)
B10b	CIFAR-10, 100	as B10, TAT with WD 0.3	−0.52 (inconcl.)	−8.13 (12/0)	−4.48 (12/0)
B11	CIFAR-100, 100	as B10, LR grid + $5{\cdot}10^{-4}$	−1.98 (12/0)	−3.61 (12/0)	−1.92 (11/1)

All primary tests are significant ($p \le 0.001$ from B9 on). The size of the advantage depends strongly on the regime: −0.8 pp with a short budget, −3.8 pp with a long budget and early stopping, −11.7 pp with AdamW. In B10 the tuning chose for TAT the largest learning rate in the grid ($2{\cdot}10^{-4}$) without weight decay; its range then grew to 36 and its 4-bit cost to +14.6 pp. The second-best TAT setting in tuning was almost as good (52.7 % vs 53.9 % validation accuracy). The −11.7 pp therefore measure TAT at one particular setting. B10b trains TAT with the two best tuning settings that use weight decay (LR 10^{-4}/WD 0.3 and $5{\cdot}10^{-5}$/WD 3.0). Weight decay improves TAT at 4 bits by 6.6 pp without a detectable fp32 cost, and BPreCap is still better by 4.48 pp (12/0, $p = 0.0005$) and 6.83 pp (12/0). Following the preregistered rule, we report the smallest margin of B9, B10 and B10b, −3.8 pp, as the cautious size. We read the B-series as "BPreCap removes the dependence of the 4-bit cost on the learning-rate and optimizer choice", not as a fixed margin.

CIFAR-100 (B11) confirms the primary result, but by a different route. BPreCap is better in fp32 (−1.98 pp, 12/0), and both forms pay the same 4-bit cost (+9.8 pp each). TAT chose LR 10^{-4}, and its range stayed at 8–9 instead of growing to 28 as in B10. On CIFAR-100 the linear control LB_tr is better than BPreCap at 4 bits (+1.03 pp for BPreCap, 1/11) and worse in fp32.

5.4 The PreCap core against a linear core

On CIFAR-10 the PreCap core is better than the linear core with the same hull in fp32, in every study that tested it: −2.95 (d100) and −4.79 pp (d200) [B3]; −2.57 pp at d200 with retuned LR [B5]; −2.14 pp with a 4× budget, constant over the run [B7]; −1.32 pp with the TAT recipe [B6]; −1.40 pp with the TAT recipe at d200 [B8]; −1.55 pp with early stopping [B9]; −1.93 pp with AdamW [B10]; and −1.55 pp on CIFAR-100 [B11, S4] (12/0 wherever reported). At 4 bits the picture is mixed: BPreCap is better at d100 on CIFAR-10 (−1.34 pp in B9, −1.36 pp in B10), LB is better at d200 (w8a4a +0.38 pp, w8a4 +3.47 pp in favor of LB) [B8], on CIFAR-100 (+1.03 pp) [B11], and on MNIST/FashionMNIST [B1]. A reading consistent with all studies: the bound carries the 4-bit robustness, and the PreCap core carries fp32 accuracy on harder data.

5.5 Common activations under the same recipe (B12, B15)

ReLU (He gain), GELU (exact, gain 1.53) and SELU (gain 1.0) were trained with the same recipe and regime as BPreCap, with LR and weight decay tuned per form on a grid that extends one step above BPreCap's choice (Table 3). The preregistered success criterion, BPreCap better than all three at w8a4a, is **not met**.

Table 3. BPreCap_tr − baseline, median paired difference of test error (pp), W/L over 12 seeds, early-stopping checkpoint. Holm over 3 (w8a4a) and over 6 (fp32, w8a4).

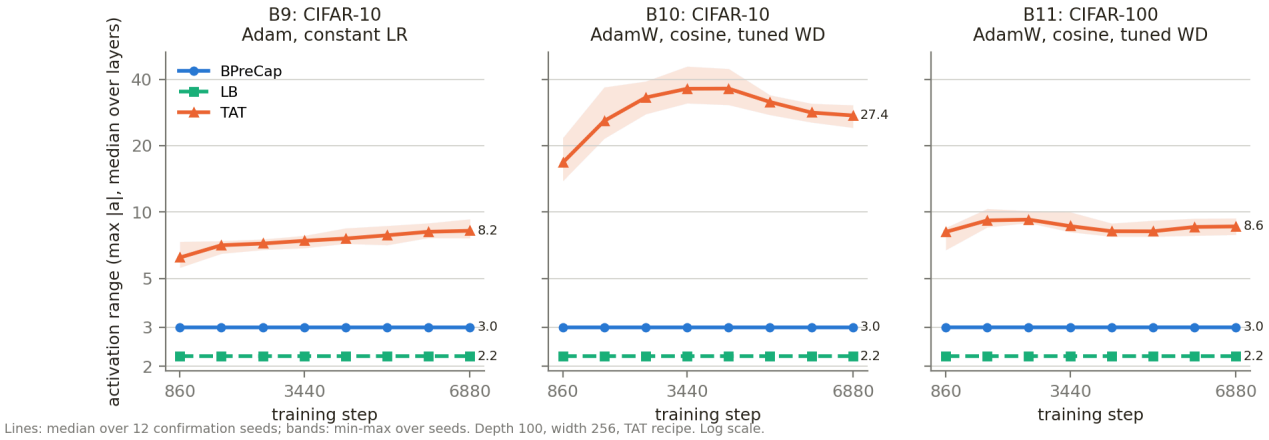
Baseline	LR / WD	fp32 of baseline	fp32	w8a4	w8a4a (primary)	4-bit cost of baseline
ReLU	$2{\cdot}10^{-4}$ / 3.0	62.5 %	−15.52 (12/0)	−35.20 (12/0)	−39.22 (12/0)	+25.1 pp
GELU	10^{-4} / 0.3	90.0 % (chance)	−43.38 (12/0)	−39.35 (12/0)	−40.34 (12/0)	–
SELU	$2{\cdot}10^{-4}$ / 0.3	47.5 %	−0.95 (12/0)	−0.75 (10/2)	−0.47 (9/3, Holm p = 0.077, inconclusive)	+2.5 pp

ReLU trains poorly in 100 layers without normalization even with orthogonal initialization, and GELU does not train at all in the confirmation runs (training loss stays at ln 10; in tuning it partly trained). This matches the depth collapse of fixed activations (Section 2.3). **SELU is a strong baseline:** in this recipe it is only 0.95 pp behind BPreCap in fp32, on par at asymmetric 4 bits, and its 4-bit cost (+2.5 pp) is slightly smaller than BPreCap's (+3.0 pp). The small 4-bit cost is therefore not specific to BPreCap. It separates BPreCap from Leaky ReLU (TAT) and ReLU, not from SELU. **At depth 200 (B15) SELU falls behind.** Both forms were tuned anew (LR {2, 5, 10, 20}·10^{-5} × WD {0, 0.3, 3.0}) and confirmed on 12 new seeds. BPreCap is better by 11.97 pp in fp32 and by 8.95 pp at w8a4a (both 12/0, p = 0.0005; symmetric 4 bits −8.61 pp, 12/0; 3 bits inconclusive). SELU trains bimodally: 5 of 12 runs barely learn (final training loss 2.2–2.3, close to ln 10), the other 7 reach 56–59 % test error, still about 10 pp behind BPreCap (46.2–47.0 % for all 12 seeds); the smallest 4-bit difference over all seeds is −7.1 pp. This is the difference predicted by the gradient gain at initialization: about 970-fold growth over 200 layers for SELU ($\chi 1 = 1.07$) against about 2.7-fold for BPreCap ($\chi 1 = 1 + 2/L$). BPreCap's own 4-bit cost rises with depth, to +7.3 pp at depth 200 (from +3.0 pp at depth 100); SELU pays +3.7 pp on a much worse fp32 level. SELU's selected LR lies at the upper edge of the grid; its neighbors were almost as good, and at depth 100 a higher LR diverged.

5.6 Replication on other hardware (C6)

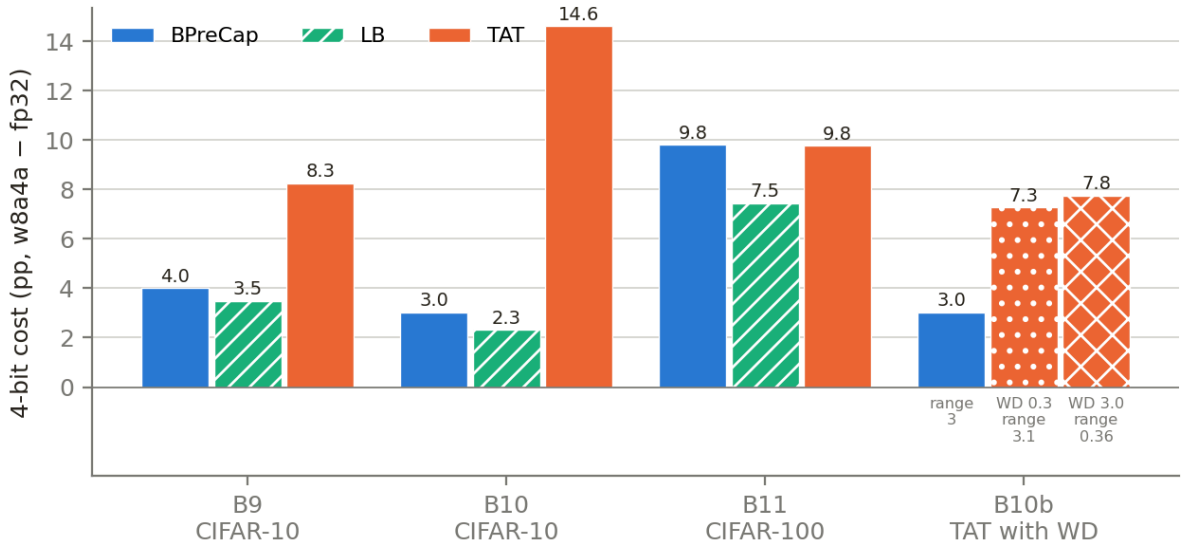
On the RTX 4070 with PyTorch, BPreCap is better than TAT with weight decay at w8a4a by 5.04 pp (12/0, p = 0.0005; MLX: 4.48 pp) and better than TAT without weight decay by 11.44 pp (12/0; MLX: 11.67 pp). The PreCap core is better than the linear core in fp32 by 1.91 pp (MLX: 1.93 pp), and BPreCap is better than SELU in fp32 by 1.13 pp (MLX: 0.95 pp). Per form, the CUDA and MLX medians differ by at most 0.48 pp, and the 4-bit costs agree (BPreCap +3.1 against +3.0 pp; TAT without weight decay +14.3 against +14.6 pp). One verdict differs while the size does not: BPreCap against SELU at 4 bits is −0.48 pp (10/2, significant) on CUDA and −0.47 pp (9/3, inconclusive) on MLX. We therefore read the 4-bit difference to SELU at depth 100 as small, not as absent or established.

5.7 What drives the 4-bit cost



Lines: median over 12 confirmation seeds; bands: min-max over seeds. Depth 100, width 256, TAT recipe. Log scale.

Figure 2). Activation range over training for BPreCap, LB and TAT in B9, B10 and B11 (median over 12 seeds, min–max band, log scale). TAT's range with weight decay (B10b) is 2.9–3.1 (WD 0.3) and 0.36 (WD 3.0), see Figure 3.



Median over 12 seeds of the per-run test-error increase at the early-stopping checkpoint.

Figure 3). 4-bit cost (w8a4a – fp32) at the early-stopping checkpoint, median over 12 seeds, for B9, B10, B11 and TAT with weight decay (B10b, with its range). Generated by `make_figures_v03.py` from the raw CSVs.

With Adam at constant LR, TAT's range grows from 6.2 to 8.2 over 6880 steps and its 4-bit cost at the early-stopping point is +8.3 pp; BPreCap stays at 3.0 with +4.0 pp [B9]. With AdamW at LR $2{\cdot}10^{-4}$, TAT's range reaches 36 (27.9 at the stopping point) and its 4-bit cost +14.6 pp; BPreCap again stays at 3.0 with +3.0 pp [B10]. On CIFAR-100 at LR 10^{-4}, TAT's range stays at 8–9 and its 4-bit cost equals that of BPreCap [B11]. This matches He et al. [25]: large adaptive learning rates promote outliers.

The absolute range is not the mechanism [B10b]. With weight decay, TAT's range at the end of training is 2.9 (WD 0.3) and 0.36 (WD 3.0), as small as BPreCap's 3.0 or smaller, yet its 4-bit cost is +9.1 and +7.8 pp against +2.9 pp for BPreCap. Leaky ReLU is positively homogeneous and per-tensor quantization with calibration is scale-invariant, so the scale of TAT's activations cannot matter by itself. What moves with TAT's 4-bit cost is the shape of the distribution: the ratio of the maximum to the 99.99th percentile falls from 1.60 (no weight decay, cost +15.5 pp) to 1.20 (WD 3.0, +7.8 pp). For the bounded forms the ratio is 1.0, since part of the units reach the bound.

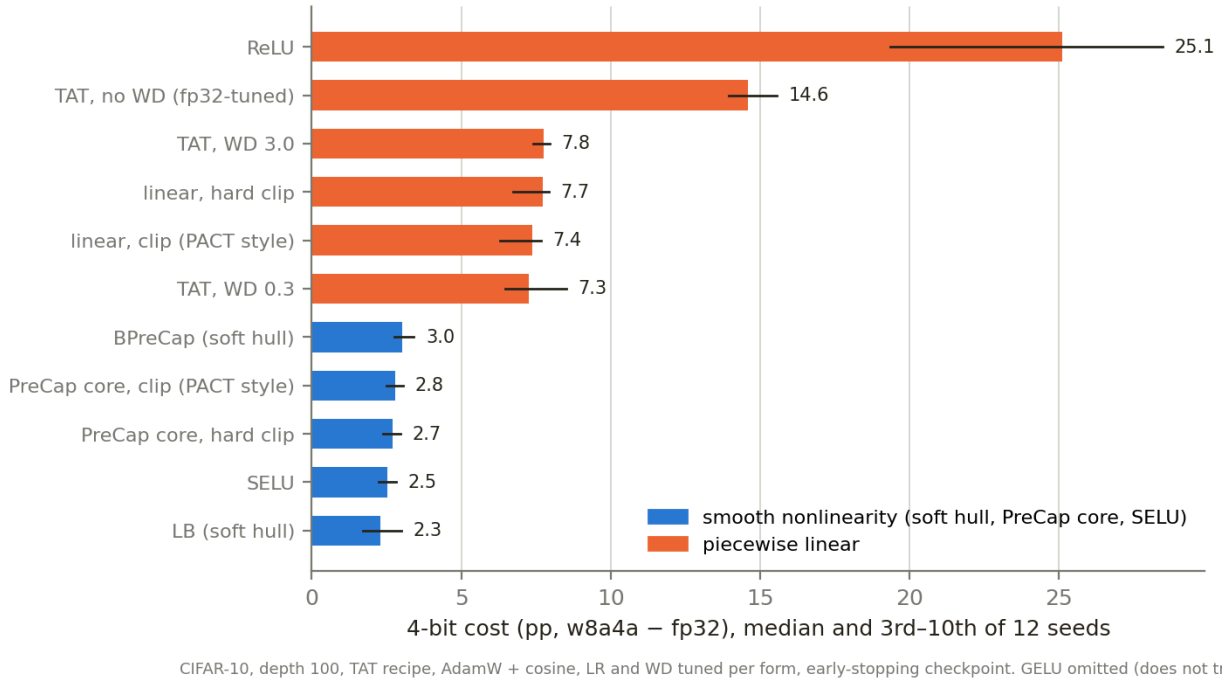


Figure 4). 4-bit cost of every form trained in the same regime (CIFAR-10, depth 100, TAT recipe, AdamW with cosine decay, LR and WD tuned per form; B10, B10b, B12, B13, B14). Forms with a smooth nonlinearity (soft hull, PreCap core, SELU) pay +2.3 to +3.0 pp; piecewise-linear forms (Leaky ReLU, ReLU, a hard clip on a linear core) pay +7.3 to +25.1 pp. This grouping was noticed after the studies and is descriptive.

Diagnosis (D1, descriptive). We retrained two seeds of six forms bit-identically to the original runs and measured, at the end of training, the shape of every layer's input distribution, the local 4-bit rounding error of each layer, and how a rounding error (or isotropic noise) injected at one layer propagates to the logits (Table 4). Two mechanisms appear. (i) **Heavy tails.** TAT's layer inputs have a median kurtosis of 12.5 with weight decay and 29 without (BPreCap 2.7, LB 2.3, SELU 2.5), and 4-bit rounding is 4.7–6.0 times less accurate per layer. (ii) **Damping over depth.** In the bounded smooth forms and SELU, an error injected in an early layer is damped to 0.1–0.2 of its relative size by the middle of the network. A hard clip on the same linear core has the same local rounding error as the soft hull (factor 1.17) but passes errors on about twice as strongly (factor 2.1–2.3, with both rounding and isotropic noise). TAT loses mainly through (i), the hard-clipped linear form through (ii). Both mechanisms have counterparts in the literature: heavy-tailed activations and kurtosis are known to drive quantization error [25, 32], and damping of compounding rounding errors is the goal of the regularizer of Scholten et al. [33]; here they arise from the form of the activation itself. Our preregistered reading rule for TAT against BPreCap gave "mixed", because the propagation factor was 1.34 in one seed and 1.59 in the other, around the threshold of 1.5; the local-error factor was far above its threshold in both. The comparisons with SELU and the hard clip were added after B12 and B13 and are descriptive only.

Table 4. D1, medians over layers, two seeds each (seed 42168 / 42169).

Form	Kurtosis	Local 4-bit error ε	Propagation gain (rounding)	Propagation gain (noise)
BPreCap	2.75 / 2.74	0.0080 / 0.0080	0.30 / 0.30	0.29 / 0.29
LB, soft hull	2.34 / 2.35	0.0063 / 0.0063	0.31 / 0.30	0.30 / 0.30
LB, hard clip	2.55 / 2.56	0.0074 / 0.0074	0.67 / 0.70	0.68 / 0.70
SELU	2.48 / 2.48	0.0103 / 0.0102	0.20 / 0.20	0.20 / 0.21
TAT, WD 0.3	12.5 / 12.6	0.0377 / 0.0381	0.40 / 0.49	0.28 / 0.29
TAT, no WD	29.0 / 29.7	0.0575 / 0.0628	0.69 / 0.63	0.39 / 0.35

What we can state from the studies themselves is the observation: without the bound, the 4-bit cost of TAT is a function of the training settings (+7 to +15 pp on CIFAR-10); with the bound it stays at about +3 pp.

On CIFAR-100, w8a4a is best around steps 3440–4300 for all forms and then worsens while fp32 still improves [B11]. Early stopping on fp32 validation therefore does not select the best 4-bit checkpoint. The same divergence of validation loss and quantization error after the learning rate decays was reported for large language models by Catalan-Tatjer et al. [29]; we observe it for all forms, weaker for the bounded ones. An oracle choice by test w8a4a (not preregistered) still leaves BPreCap ahead of TAT on CIFAR-10 (−3.06 pp in B9, −10.1 pp in B10).

5.8 Summary of version 0.3

Claim	Evidence	Status
One form trains at depth 200 and keeps 4-bit robustness	B1, B5	confirmed (fp32 cost ~1 pp at d200)
Soft better than hard at the same bound, Gaussian init	B2, B4	confirmed, small (0.8–1.8 pp)
Soft better than hard in the TAT recipe	B13	linear core: confirmed (−4.7 pp at 4 bits); PreCap core: inconclusive
Piecewise-linear forms pay more at 4 bits than smoothly saturating ones	B10b, B12, B13, descriptive	consistent pattern (ReLU +25, TAT +7 to +15, hard-clipped linear +7.7 vs. +2.3 to +3.0 pp); not tested directly
Fixed soft bound better than learned hard clip (PACT style)	B14	not testable: the bound did not learn under AdamW; equivalent to B13
PreCap core better than linear core in fp32 (CIFAR)	B3, B5, B6, B7, B9, B11	confirmed
PreCap core better than linear core at 4 bits	B1, B8, B9, B10, B11	mixed, depends on data and depth
On par with TAT in fp32, same recipe, CIFAR-10	B8, B9, B10	confirmed (inconclusive = on par)
Better than TAT at 4 bits, same recipe	B6, B8, B9, B10, B10b, B11	confirmed in 6/6; size depends on regime and TAT's settings (cautious size −3.8 pp)
The fixed range explains the 4-bit advantage	B10b, descriptive	not supported: TAT with weight decay has the same range and still pays more
Mechanism: heavy tails (TAT) and missing damping (hard clip)	D1, descriptive	observed in 2 seeds each; preregistered reading for TAT "mixed" (propagation factor at threshold)
Better than ReLU and GELU, same recipe	B12	confirmed (both train poorly at depth 100)
Better than SELU, same recipe, depth 100	B12	fp32 yes (−0.95 pp); 4 bits inconclusive; SELU's 4-bit cost is as small
Better than SELU, same recipe, depth 200	B15	confirmed: fp32 −11.97 pp, 4 bits −8.95 pp (12/0 each); SELU trains bimodally
Results independent of framework and hardware	C6 (PyTorch/CUDA, RTX 4070)	confirmed: same verdicts against TAT and LB, medians within 0.5 pp
Trains only with small LR	B10, B11, B13–B15 tuning	observed

6 Package

Version 0.3.0 of the package adds the module `BPreCap(depth=100|200)` and the soft hull `Lagma(M, rho=0.8)` for PyTorch and MLX, the functional hull `lagma_hull(y, M, rho=0.8)`, the reference functions `bprecap_ref` and `lb_ref`, and the constants of Table 1 (`BOUNDED`). The tests check the constants against the study code (10^{-12}), the moments and $\chi 1$, the hull (identity, C^1 at the knee, bounded, monotonic), gradients (gradcheck), and MLX against PyTorch ($2{\cdot}10^{-6}$).

Fused kernels. The studies used the reference implementation, a chain of elementwise operations; on the RTX 4070 this made a BPreCap run about seven times slower than a SELU run (C6). Version 0.3.0 therefore adds fused kernels for BPreCap and Lagma, forward and backward, in Metal (`mx.fast.metal_kernel`) and in CUDA (`torch.cuda.jiterator`); the original PreCap kernels remain. They are the default (`backend='ref'` selects the reference). Against float64 they agree to $1.3{\cdot}10^{-6}$ forward and to $2.2{\cdot}10^{-6}$ in the gradient (relative to $1 + |g|$), including edge values up to $\pm 10^4$, and pass gradcheck as well as float16/bfloat16 and shape tests (`kernels/test_bprecap_mlx.py`, `kernels/test_bprecap_cuda.py`). Time for forward and backward on 16M elements:

	BPreCap kernel	BPreCap ref.	Lagma kernel	Lagma ref.	SELU
MLX, M1 Pro	3.1 ms	60.7 ms	2.0 ms	21.5 ms	10.0 ms
CUDA, RTX 4070	1.05 ms	19.0 ms	1.04 ms	8.7 ms	0.73 ms

With the kernels, BPreCap is faster than SELU in MLX and takes about 1.4 times as long as PyTorch's native SELU on CUDA. The studies themselves were run with the reference implementation, so their results do not depend on the kernels.

7 Limitations

- Only fully connected networks of width 256, depths 100 and 200. No CNNs, no Transformers, no residual networks. Whether the bound helps when skips and normalization are present is open.
- MNIST, FashionMNIST, CIFAR-10, CIFAR-100. The absolute accuracy of an MLP on CIFAR is low (about 50 % on CIFAR-10, 25 % on CIFAR-100); the studies compare forms, not the state of the art on these datasets.
- Only post-training quantization, simulated, per tensor for activations. Quantization-aware training, per-channel activation scales and real int4 hardware were not tested.
- TAT is our replication with the official parameters, trained with Adam/AdamW instead of K-FAC. A non-diagonal optimizer such as SOAP, which reduces outliers [25], was not tested.
- Hyperparameters were tuned on 2 tuning seeds (B10–B14). In B10 TAT's LR, in B11 BPreCap's and LB's LR lie at the edge of the grid.
- BPreCap diverges at peak LR $2{\cdot}10^{-4}$ (B10, B11 tuning), where TAT still trains. The hard-clipped variants do too (B13, B14), so the soft hull is not the cause; the cause is not established.
- $M = 3$ and $\rho = 0.8$ were fixed, not optimized.
- No fair comparison with PACT: a learnable bound did not move under AdamW (B14).
- SELU matches BPreCap at 4 bits at depth 100 under the same recipe (B12); at depth 200 it falls clearly behind (B15). Other depths and recipes for SELU were not tested.
- BPreCap's 4-bit cost grows with depth (+3.0 pp at depth 100, +7.3 pp at depth 200), and at depth 200 it diverges already at peak LR 10^{-4}.
- The size of the 4-bit advantage over TAT varies by an order of magnitude across regimes (0.8 to 11.7 pp). Only its direction is stable.
- The studies ran on Apple MLX; the replication on PyTorch/CUDA (C6) covers the main comparisons at depth 100 on CIFAR-10, not CIFAR-100 (B11) or depth 200 (B15). It uses the same data, initial weights and batch order, so framework, kernels and hardware are independent, not the experimental design.
- The work was done by a single independent researcher on consumer hardware, with AI assistance for code, analysis and writing (see acknowledgments).

8 Conclusion

A soft, fixed bound around the depth-calibrated PreCap core resolves the trade-off of version 0.2 for deep MLPs without normalization or skips: the same form trains at depths 100 and 200 and keeps a small, fixed activation range. Under TAT's recipe it is on par with TAT in fp32 on CIFAR-10, better on CIFAR-100, and better at 4 bits in every tested regime, by a margin that depends on TAT's learning rate and weight decay (−3.8 to −11.7 pp on CIFAR-10). The absolute range does not explain why the bounded forms pay so little at 4 bits; a diagnosis points to light-tailed activations and to damping of

rounding errors over depth, both shared by SELU. At depth 100 the advantage at 4 bits is an advantage over piecewise-linear activations such as TAT's Leaky ReLU, not over SELU; at depth 200, where SELU's fixed gradient gain makes training unstable, BPreCap is also clearly better than SELU. The bound appears to carry the 4-bit robustness and the PreCap core the fp32 accuracy on harder data. Whether the bound helps in convolutional networks and Transformers is the main open question.

Acknowledgments. The functions PreCap, SPreCap, the depth family and the soft hull (Lagma) are the author's ideas, and all experiments were run by the author's setup. Implementation, statistical analysis, literature research and writing were developed with the help of Claude Code (Anthropic). See `docs/Danksagung_und_Werkzeuge.md`.

Reproducibility. For every study of version 0.3, the repository contains the preregistration, the frozen scripts with SHA-256 hashes and freeze times, raw results per run, the analysis script and its output, and the report: `[repo path]/`. Preregistrations are published unchanged; corrections are noted in the reports.

References

[1] Klambauer, Unterthiner, Mayr, Hochreiter. Self-Normalizing Neural Networks. NeurIPS 2017. https://arxiv.org/abs/1706.02515

[2] Chen, Zhao, Deng, Ding, Wen, Li, Xie. Large-scale self-normalizing neural networks (bSELU). Journal of Automation and Intelligence 3, 101–110, 2024. https://www.sciopen.com/article/10.1016/j.jai.2024.05.001

[3] Hayou, Doucet, Rousseau. On the Impact of the Activation Function on Deep Neural Networks Training. ICML 2019. https://arxiv.org/abs/1902.06853

[4] Lu, Gould, Ajanthan. Bidirectionally Self-Normalizing Neural Networks. 2020/2021. https://arxiv.org/abs/2006.12169

[5] Zaied. SaluNet: Enabling Total Plasticity in Normalization-Free Networks. 2026. https://arxiv.org/abs/2606.02927

[6] Zhu, Chen, He, LeCun, Liu. Transformers without Normalization (DyT). CVPR 2025. https://arxiv.org/abs/2503.10622

[7] Chen, Lu, Zhu, Sun, Liu. Stronger Normalization-Free Transformers (Derf). 2025. https://arxiv.org/abs/2512.10938

[8] Clevert, Unterthiner, Hochreiter. Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs). ICLR 2016. https://arxiv.org/abs/1511.07289

[9] Bukva, de Gier, Grosvenor, Jefferson, Schalm, Schwander. Criticality versus uniformity in deep neural networks. 2023. https://arxiv.org/abs/2304.04784

[10] Liu, Di. TanhExp: A Smooth Activation Function with High Convergence Speed for Lightweight Neural Networks. IET Computer Vision. https://arxiv.org/abs/2003.09855

[11] Kunc, Kléma. Three Decades of Activations: A Comprehensive Survey of 400 Activation Functions for Neural Networks. 2024. https://arxiv.org/abs/2402.09092

[12] Zhang, Botev, Martens. Deep Learning without Shortcuts: Shaping the Kernel with Tailored Rectifiers. ICLR 2022. https://arxiv.org/abs/2203.08120

[13] Martens, Ballard, Desjardins, Swirszcz, Dalibard, Sohl-Dickstein, Schoenholz. Rapid training of deep neural networks without skip connections or normalization layers using Deep Kernel Shaping. 2021. https://arxiv.org/abs/2110.01765

[14] Li, Nica, Roy. The Neural Covariance SDE: Shaped Infinite Depth-and-Width Networks at Initialization. NeurIPS 2022. https://proceedings.neurips.cc/paper_files/paper/2022/hash/45fc4a0da7e7f6fbabaabe2d20a441d1-Abstract-Conference.html

[15] Nagel, Fournarakis, Amjad, Bondarenko, van Baalen, Blankevoort. A White Paper on Neural Network Quantization. 2021. https://arxiv.org/abs/2106.08295

[16] Choi, Wang, Venkataramani, Chuang, Srinivasan, Gopalakrishnan. PACT: Parameterized Clipping Activation for Quantized Neural Networks. 2018. https://arxiv.org/abs/1805.06085

[17] Chen, Deng, Wang, Li, Xie. A Comprehensive and Modularized Statistical Framework for Gradient Norm Equality in Deep Neural Networks. IEEE TPAMI 2020. https://arxiv.org/abs/2001.00254

[18] Pennington, Schoenholz, Ganguli. Resurrecting the sigmoid in deep learning through dynamical isometry: theory and practice. NeurIPS 2017. https://arxiv.org/abs/1711.04735

[19] Ji, Martens, Zheng, Zhou, Moghadam, Zhang, Saratchandran, Lucey. Cutting the Skip: Training Residual-Free Transformers. arXiv 2025. https://arxiv.org/abs/2510.00345

[20] Ji, Martens, Lucey. Should We Skip Diffusion? arXiv 2026. https://arxiv.org/abs/2610.07002

[21] Zhou, Wu, Ni, Zhou, Wen, Zou. DoReFa-Net: Training Low Bitwidth Convolutional Neural Networks with Low Bitwidth Gradients. 2016. https://arxiv.org/abs/1606.06160

[22] Zhao, Liu, Li. Efficient Integer-Arithmetic-Only Convolutional Neural Networks. 2020. https://arxiv.org/abs/2006.11735

[23] Banner, Nahshan, Soudry. Post training 4-bit quantization of convolutional networks for rapid-deployment. NeurIPS 2019. https://arxiv.org/abs/1810.05723

[24] Park, Jang, Park. Symmetry Regularization and Saturating Nonlinearity for Robust Quantization. ECCV 2022. https://arxiv.org/abs/2208.00338

[25] He, Noci, Paliotta, Schlag, Hofmann. Understanding and Minimising Outlier Features in Neural Network Training. NeurIPS 2024. https://arxiv.org/abs/2405.19279

[26] Tarawneh. Depth Laws for the Precision Floor of Trained Neural Networks. 2026. https://arxiv.org/abs/2609.32060

[27] Ji, Sabanayagam, Moghadam, Saratchandran, Lucey. The Quantization Benefits of Residual-Free Transformers. arXiv 2026 (under review). https://arxiv.org/abs/2605.25880

[28] Sun, Jiang, Chee, Devos, Botteldooren. Batch Normalization-Free Fully Integer Quantized Neural Networks via Progressive Tandem Learning. arXiv 2025. https://arxiv.org/abs/2512.16476

[29] Catalan-Tatjer, Ajroldi, Geiping. Training Dynamics Impact Post-Training Quantization Robustness. 2025. https://arxiv.org/abs/2510.06213

[30] Kavuri, Babu-PK, Ellapan, Maddu, Deshpande. SCULPT: Training Edge Vision Models for Post-Training Quantization Readiness. 2026. https://arxiv.org/abs/2609.01743

[31] Blumenfeld, Gilboa, Soudry. A Mean Field Theory of Quantized Deep Networks: The Quantization-Depth Trade-Off. NeurIPS 2019. https://arxiv.org/abs/1906.00771

[32] Shkolnik, Chmiel, Banner, Shomron, Nahshan, Bronstein, Weiser. Robust Quantization: One Model to Rule Them All. NeurIPS 2020. https://arxiv.org/abs/2002.07686

[33] Scholten, Lawrence, Hensman, Günnemann, Curth, Grazzi. Activation Denoising: A Robustness View on Parallel vs Sequential LLM Quantization. arXiv 2026. https://arxiv.org/abs/2610.07522

[34] Schmitt. PreCap: A Bounded, Self-Normalizing Activation Function that Matches SELU in Deep Normalization-Free MLPs (preprint v0.1). Zenodo 2026. https://doi.org/10.5281/zenodo.23676200

[35] Schmitt. PreCap: Bounded, Self-Normalizing Activations for Deep Normalization-Free MLPs – Depth Calibration, a Comparison with TAT, and 4-Bit Robustness (preprint v0.2). Zenodo 2026. https://doi.org/10.5281/zenodo.23757287

[36] Huang, Ni, Yang. Efficient Quantization for Neural Networks with Binary Weights and Low Bitwidth Activations. AAAI 2019. https://doi.org/10.1609/aaai.v33i01.33013854

[37] Cai, Tang. Beyond ReLU and GELU: SoftCap Bounded Activations for Stability, Sparsity, and Quantization (preprint v3). Zenodo 2026. https://zenodo.org/records/19622248

[38] Park, Lee, Yoon, Hwang, Kang. Outlier-Safe Pre-Training for Robust 4-Bit Quantization of Large Language Models. 2025. https://arxiv.org/abs/2506.19697

[39] Chavan, Lele, Bamba, Dayal, Raghunathan, Gupta. S2D: Selective Spectral Decay for Quantization-Friendly Conditioning of Neural Activations. 2026. https://arxiv.org/abs/2602.14432

[40] Wu, Li, Chen, Shi. Training and Inference with Integers in Deep Neural Networks. ICLR 2018. https://arxiv.org/abs/1802.04680

[41] Li, An, Zang, Ye, Xie, Qiu, Shen, He, Wang. InfoQuant: Shaping Activation Distributions for Low-Bit LLM Quantization. 2026. https://arxiv.org/abs/2605.26175

[42] Zhao, Dong, Keutzer. Analysis of Quantization on MLP-based Vision Models. 2022. https://arxiv.org/abs/2209.06383

[43] Yang, Chi, Deng, Yan, Gao, Li. Towards Efficient Full 8-bit Integer DNN Online Training on Resource-limited Devices without Batch Normalization. 2021. https://arxiv.org/abs/2105.13890

[44] LeCun, Bottou, Orr, Müller. Efficient BackProp. In: Neural Networks: Tricks of the Trade, Springer 1998.

[45] Price, Daultry Ball, Lam, Jones, Tanner. Deep Neural Network Initialization with Sparsity Inducing Activations. ICLR 2024. https://arxiv.org/abs/2402.16184

[46] Sandler, Howard, Zhu, Zhmoginov, Chen. MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR 2018. https://arxiv.org/abs/1801.04381